Paiflow × «EVENT_PARTNER» — Event Primer

Template for review · «UPDATED_DATE» · «VERSION»

Build an app that solves a real problem with automated payments through Paiflow. Your app handles the customer experience; your flow handles when money moves and where it goes. «EVENT_THEME_OR_OPEN_CHALLENGE»

01 At a glance

Item	Details
Date and time	«EVENT_DATE» · «START_TIME»–«END_TIME» · «TIMEZONE»
Venue or online access	«VENUE_AND_ROOM_OR_MEETING_LINK»
Format and eligibility	«IN_PERSON_ONLINE_OR_HYBRID» · «WHO_CAN_JOIN»
Teams	«MAX_TEAMS» teams · «MIN_TEAM_SIZE»–«MAX_TEAM_SIZE» people
Registration deadline	«REGISTRATION_DEADLINE_WITH_TIMEZONE»
Submission deadline	«SUBMISSION_DEADLINE_WITH_TIMEZONE»
Prize pool	«CURRENCY_AND_TOTAL_OR_NO_PRIZES»
Network	Stellar testnet; testnet funds have no real-money value

02 Preparation and integration timeline

Before the event — plan: choose a problem, sketch your app and register by «REGISTRATION_DEADLINE_WITH_TIMEZONE».

Before the event — prepare and integrate: set up wallets and testnet funds using the Paiflow guide. USDC needs its exact trustline and separate funding; Friendbot supplies XLM only. «EARLY_DEPLOYMENT_API_INTEGRATION_AND_TEST_PAYMENT_POLICY».

Onboarding: join the online onboarding session to learn the event mechanics and ask the organisers questions: «ONBOARDING_DATE_TIME_TIMEZONE_AND_ACCESS_OR_NO_SESSION». «ONBOARDING_DEMO_SETUP_HELP_AND_PREPARATION_DETAILS». Follow the early-integration policy above when building and testing around this session.

During the event — finish and demonstrate: use «EVENT_BUILD_WINDOW_WITH_TIMEZONE» to complete integration, verify a payout, collect evidence and prepare your pitch. Early preparation does not replace the qualifying payout requirement below.

02 Preparation and integration timeline

Submission deadline: «SUBMISSION_DEADLINE_WITH_TIMEZONE» via «SUBMISSION_URL». Submit your evidence on time, even if app polishing continues. «LATE_SUBMISSION_AND_TECHNICAL_FAILURE_POLICY».

03 Event program

Prepare before the event, use the event's build sessions to finish and demonstrate your app, then submit before the deadline. The preparation timeline above states what is permitted early.

The schedule lists event-day activities. Read it together with the preparation permissions above.

Hackathon schedule

All times are in **«TIMEZONE»**. «BREAK_AND_REMOTE_ATTENDANCE_DETAILS»

Time	Activity	What happens
«CHECK_IN_START_END»	Check-in and briefing	Confirm attendance, rules and support access
«BUILD_SESSION_1_START_END»	Build and integration	Work toward a confirmed payout
«BREAK_START_END»	Break	«BREAK_PROVISIONS_AND_OPTIONAL_CODING_RULE»
«BUILD_SESSION_2_START_END»	Build and integration	Finish the app and prepare evidence
«SUBMISSION_WINDOW»	Submission	Submit by «SUBMISSION_DEADLINE»; «OVERLAP_WITH_BUILD_SESSION»
«PITCHING_START_END»	Pitching	«TEAM_COUNT» × 5 minutes (2-minute pitch/demo + 3-minute Q&A), plus «TRANSITION_AND_BUFFER_MINUTES» minutes
«JUDGING_START_END»	Judging	Verify evidence and finalise scores
«AWARDS_AND_CLOSING_START_END»	Awards and closing	Announce results and next steps

04 The challenge

Create a working app with Paiflow at the centre of its payment experience. Choose a real user and a problem where receiving, routing or releasing money is essential to the solution. Your app provides the experience people use; Paiflow runs the payment flow that makes it work. «THEME_CONSTRAINTS_OR_OPEN_THEME».

04 The challenge

What your team needs to build

1. **Choose the problem and user.** Explain who pays, why they pay, who receives the money and what your app helps them do.
2. **Design and deploy your Paiflow flow.** Choose when money moves, any conditions it must meet and where it goes. Configure recipients and the asset before deployment.
3. **Connect your app to that flow.** Use the SDK or HTTP API so the payment is part of the app's main user journey. Show payment status and the confirmed outcome in your app.
4. **Demonstrate the complete journey.** Use your team's own deployment, show the customer experience and verify that money reaches the intended destination. Keep the deployment ID and transaction hash available.

Paiflow turns your visual flow into Stellar smart contracts. One deployment can serve a business or group and accept payments from multiple customers. The SDK and HTTP API operate existing deployments; they do not create a separate deployment for every app user.

Optional inspiration — choose your own idea

> **These are examples, not assigned challenges or a list you must choose from.** You may propose a different app within the event theme. Choose the flow that fits your user's problem; splitting payments is optional. A useful app can pay one recipient, route a payment by a condition or hold funds until a date.

For more inspiration, explore previous Paiflow hackathon winners. Use them as a starting point to develop your own idea for your users.

Confirm event support before choosing an example involving conditions. The app features described below, such as tickets and order status, are built by your team.

Example app	What your app helps users do	Paiflow's role
Workshop ticketing	Pay an organiser and receive a ticket after payment confirmation	On Receive → Pay one organiser
Service booking	Pay a provider and track a confirmed booking	On Receive → Pay one provider
Donation routing	Send each donation to one of two beneficiaries based on that payment's amount	On Receive → Amount condition → Pay on each branch
Savings lock	Deposit funds and track when they become available	On Receive → Time condition, with Hold before the deadline and Pay after it

> **Not sure whether a Paiflow feature is available for your idea?** Ask the organisers at «TECHNICAL_SUPPORT_CHANNEL» before you start building.

05 Pitching requirements and format

Each team has **two minutes for the pitch and demo, followed by three minutes for Q&A. Focus on the demo:** spend most of your two minutes showing the app and its payment flow in action.

05 Pitching requirements and format

Cover the points below briefly, using **one or two sentences for each**. They do not need dedicated slides. You can explain them during your demo instead of using a presentation deck; slides are optional.

1. **Problem:** describe the intended user and their problem. Keep claims specific and proportionate; do not overstate the problem.
2. **Solution:** explain what your app does and how it addresses the problem.
3. **Impact on Stellar:** explain how your app uses Stellar and what useful activity or adoption it could bring. Distinguish demonstrated results from future potential.
4. **Paiflow integration:** show your flow and explain its role in the app.
5. **Demo:** show the customer experience, payment and confirmed payout to the intended recipients. Keep the deployment ID and transaction hash available for verification.

Have your app and any slides ready before your turn and start promptly.
«ORDER_TIMER_SETUP_AND_FALLBACK_RECORDING_RULES»

06 Prizes

Qualification: «CONFIRMED_TEAM_DEPLOYMENT_AND_VERIFIABLE_PAYOUT_REQUIREMENTS».
«NON_QUALIFYING_TEAM_PITCH_AND_PRIZE_POLICY»

Judging criteria:

Criterion	Weight
Problem and user	15%
Solution and creativity	20%
Impact on Stellar	10%
Paiflow integration	30%
Works end to end	25%

Publish any live-versus-recorded demo scoring difference here: «DEMO_SCORING_POLICY». Full internal judge instructions must apply these published rules.

PRIZE POOL

«CURRENCY_AND_TOTAL_OR_NO_PRIZES»

07 Event support and coordination

Submission checklist

Submit at **«SUBMISSION_URL»** by **«DEADLINE_AND_TIMEZONE»**: team details, project summary, «SOURCE_OR_DEMO_LINKS», deployment ID, payout transaction hash and «DEMO_REQUIREMENTS». «PAYOUT_EVIDENCE_REQUIREMENTS». Never submit tokens, private keys or signed envelopes.

Resources and support

Resource	Link or contact
Registration and submission	«REGISTRATION_URL» · «SUBMISSION_URL»
Paiflow app and setup guide	«APP_URL» · «PAIFLOW_GUIDE_URL»
Windows development setup	«WINDOWS_SETUP_GUIDE_URL»
SDK integration	«SDK_README_URL» · «SDK_METHOD_AND_TYPE_REFERENCE_URL»
HTTP integration	«API_README_URL»; includes OpenAPI and Postman
SDK payment example	«STARTER_URL»
Sample applications	«SAMPLE_SOURCE_AND_DEMO_URLS»
Tutorials and AI context	«TUTORIAL_URL» · «AI_CONTEXT_URL»
Support and announcements	«SUPPORT_CONTACT_AND_AVAILABILITY»

What to prepare and bring

Bring «EQUIPMENT_INTERNET_AND_POWER_REQUIREMENTS». Install the versions listed in the starter's current instructions and prepare a GitHub account and supported testnet wallet.
«MEALS_ACCESSIBILITY_AND_VENUE_OR_REMOTE_PROVISIONS»

Shared rules: use Paiflow for the core payment flow; use testnet only. Never expose API tokens, webhook secrets, private keys or signed envelopes in screenshots or demos.
«AI_OPEN_SOURCE_AND_EXISTING_WORK_DISCLOSURE_POLICY». No API or relayer load-testing or spam.
«CONDUCT_PRIVACY_RECORDING_AND_DISPUTE_CONTACT».
«HOW_RULE_CHANGES_WILL_BE_ANNOUNCED»